

2. LOW-LEVEL DESIGN (LLD)

1. Modules

1. ingest.py

- Loads PDF
- Splits into chunks
- Stores embeddings

2. retriever.py

- Loads ChromaDB
- Retrieves top-k relevant chunks

3. graph.py

Implements workflow using LangGraph:

Nodes:

- input_node → retrieves context
- router_node → decides path
- answer_node → generates answer
- hitl_node → human intervention
- output_node → formats response

4. main.py

- CLI interface
- Takes input
- runs graph

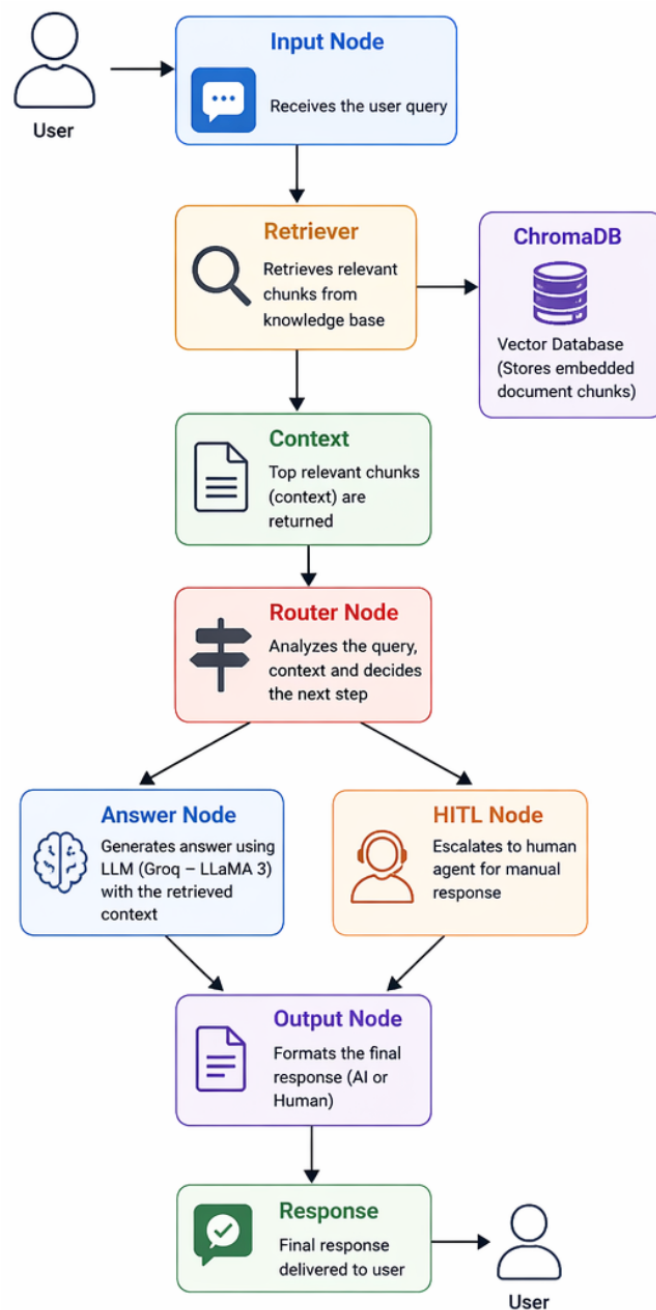
2. Data Structure

SupportState:

- user_query
- retrieved_docs
- context
- intent
- confidence
- answer
- source
- needs_human

Field	Purpose
user_query	user input
context	retrieved text
intent	answer or escalate
confidence	high/low
answer	final output
source	llm or human
needs_human	routing decision

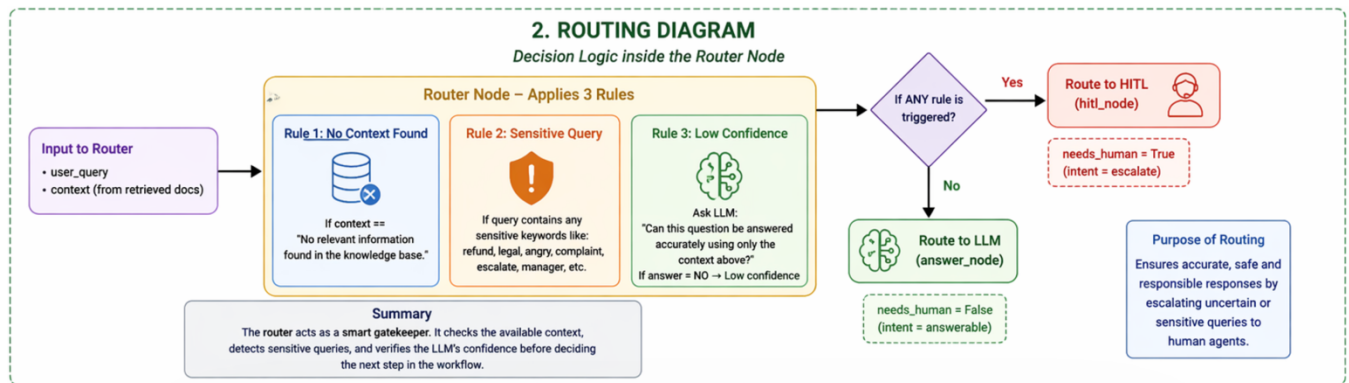
3. Workflow



4. Routing Logic

Escalation happens if:

- No context found
- Sensitive keywords detected
- Low confidence from LLM



5. HITL Design

- Triggered when escalation required
- Human inputs response via CLI
- Response returned to user

6. API / Interface

Input: User text query

Output: Answer string with label

7. Error Handling

1. Missing PDF

It is handled in `ingest.py`

```
if not os.path.exists(pdf_path):  
    raise FileNotFoundError(...)
```

When this error occurs:

- Program stops immediately
- User gets clear error message

The system validates the presence of the knowledge base before ingestion. If the PDF is missing, execution is halted with an informative error message to prevent invalid processing.

2. No Relevant Chunks Found

It is handled in `retriever.py`

```
if not docs:
    return "No relevant information found..."
```

When this error occurs:

- Context becomes empty
- Router detects it

Then in `router_node`:

```
no_context = context == "No relevant information found..."
```

If no relevant information is retrieved, the system avoids generating unreliable responses and instead escalates the query to a human agent.

3. API Failures (Groq / LLM)

It is handled in `main.py`

```
except Exception as e:
    print("Error:", e)
```

When this error occurs:

- Error is caught
- System does not crash silently
- User is notified

The system includes exception handling to capture API-related failures such as model errors or connectivity issues, ensuring graceful degradation.

4. Empty Query

It is handled in `main.py`

```
if not user_input:
    continue
```

When this error occurs:

- Ignores empty input
- Prevents unnecessary processing

The system validates user input and skips empty queries to maintain efficiency and avoid unnecessary computation.